Lab 1: The Manual Scheduler

Course: Parallel Architectures
Time Limit: 120 Minutes
Constraint: Use OpenMP to manually manage threads (SPMD Pattern).
Not Allowed: #pragma omp for and #pragma omp parallel for

Mission Briefing

Subject: We are upgrading the physics engine from 1D vectors to 2D fields (N × N arrays).

Problem: The automatic compiler optimization (#pragma omp parallel for) is "broken" for this mission. You must manually calculate which rows of the matrix each thread should process.

Goal: Implement Matrix Addition:

C[i][j] = A[i][j] + B[i][j]

Allowed resources (from lecture slides):

- SPMD Pattern: Single Program, Multiple Data
- Manual thread info:
    - omp_get_thread_num()
    - omp_get_num_threads()
- Manual decomposition math

Deliverables (What to Submit)

Submit

1. matrix_lab.c
2. One file (results.txt) containing:
    A. Level 1 time (serial baseline)
    B. Level 2 time (manual block scheduling) + speedup
    C. Level 3 time (manual cyclic scheduling) + speedup
    D. Your answers to the analysis questions in Levels 2–5

Setup

Step 1: Create your file

Create a file named: matrix_lab.c

Step 2: Use this starter code

Download the attached file and rename it to matrix_lab.c. You may add helper functions, but do not remove the overall structure. The following is the content of the file:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4000

double* alloc_matrix(int n) {
    double *m = (double*) malloc((size_t)n * (size_t)n * sizeof(double));
    if (!m) {
        printf("Allocation failed!\n");
        exit(1);
    }
    return m;
}

void fill_matrix(double *m, int n, double val) {
    for (int i = 0; i < n*n; i++) m[i] = val;
}

int main(int argc, char **argv) {

    // Optional: run like ./matrix_lab 6
    if (argc >= 2) {
        int t = atoi(argv[1]);
        if (t > 0) omp_set_num_threads(t);
    }

    double *A = alloc_matrix(N);
    double *B = alloc_matrix(N);
    double *C = alloc_matrix(N);
```

```
fill_matrix(A, N, 1.0);
fill_matrix(B, N, 1.0);
fill_matrix(C, N, 0.0);

// ========================================================
// LEVEL 1: SERIAL BASELINE
// ========================================================

// TODO: Implement Level 1 here

// ========================================================
// LEVEL 2: MANUAL BLOCK DECOMPOSITION
// ========================================================

// TODO: Implement Level 2A here

// TODO: Implement Level 2C here

// ========================================================
// LEVEL 3: CYCLIC DECOMPOSITION
// ========================================================

// TODO: Implement Level 3 here

// ========================================================
// LEVEL 4: DEPENDENCY TRAP
// ========================================================

// TODO: Implement Level 4A here

// TODO: Implement Level 4B here
```

```
    // ========================================================
    // LEVEL 5: BARRIER SYNCHRONIZATION
    // ========================================================

    // TODO: Implement Level 5A here

    // TODO: Implement Level 5B here

    free(A);
    free(B);
    free(C);
    return 0;
}
```

Step 3: Compile and run

Compile:

```
gcc -fopenmp matrix_lab.c -o matrix_lab
```

Run (example with 6 threads):

```
./matrix_lab 6
```

Level 1: The Baseline

Objective: Create the serial implementation to measure "speedup" later.

Requirements

1. Use the provided matrices A, B, and C.
2. Ensure A and B contain known simple values (already filled with 1.0 in the starter code).
3. Compute matrix addition in serial:
   C[i][j] = A[i][j] + B[i][j]
4. Measure the runtime using omp_get_wtime() and print the elapsed time.

Correctness check (required)

Print the last element:

- C[N-1][N-1]

Expected result (since A and B are filled with 1.0):

- 2.0

Notes

- Your matrices are stored as 1D arrays. You must access elements correctly using an indexing strategy that maps (i, j) to a single position in memory.
- Do not use OpenMP parallelism in Level 1.

What to write in results.txt:

- Level 1 time
- The printed value of C[N-1][N-1]

Level 2: Manual Block Decomposition

Objective: Manually divide the matrix rows among threads using SPMD.

Constraint reminder:

- You must NOT use omp for or parallel for
- You must use omp parallel and manual scheduling

Task A: Manual Block Scheduling (first attempt)

Inside a parallel region, each thread must:

1. Get:
    - its thread ID (id)
    - the total number of threads (T)
2. Compute its row range using:
    - chunk = N / T
    - start_row = id * chunk
    - end_row = start_row + chunk
3. Compute matrix addition only for its assigned rows.

Output requirement

After the parallel region:

- Print the elapsed time for Level 2
- Print C[N-1][N-1]

Task B: The "Odd Number" Trap

Run your code using a thread count that does NOT divide N evenly (example: 6).

Answer in results.txt:

- Is C[N-1][N-1] correct?
- If not, explain what happened and why.

Task C: Fix the remainder bug

Modify your scheduling logic so that all rows are computed even if N % T != 0.

Hint: A common fix is to give the "extra leftover rows" to one thread (often the last thread).

Re-run with the same thread count (example: 6) and confirm:

- C[N-1][N-1] is correct

What to write in results.txt:

- Level 2 time (after fixing)
- Your explanation of the bug and your fix

Level 3: Cyclic Decomposition

Objective: Implement round-robin scheduling (cyclic distribution of rows).

Requirements

Inside a parallel region:

1. Each thread should compute rows in a round-robin pattern
2. Measure runtime using omp_get_wtime().

Output requirement

- Print Level 3 elapsed time
- Print C[N-1][N-1]

Analysis question (answer in results.txt)

Does cyclic decomposition naturally handle the remainder (when N is not divisible by T)? Why?

What to write in results.txt:

- Level 3 time
- Speedup of Level 3 vs Level 1
- Your answer to the remainder question

Level 4: The Dependency Trap

Objective: Attempt to parallelize a loop with a dependency and explain why it fails.

New kernel:
C[i][j] = C[i][j] + C[i-1][j]

Task A: Serial correct version

1. Reinitialize C with a known value (example: all ones).
2. Implement the loop in serial.
3. Print one "checkpoint" value (example: C[1000][0]) to use as a correct reference.

Task B: Naive parallel attempt (manual block scheduling)

1. Reinitialize C again with the same known values.
2. Apply your Level 2 manual block decomposition to this kernel.
3. Run with 4 threads.
4. Print the same checkpoint value again.

Debugging questions (answer in results.txt)

- Why is the output wrong?
- What happens at the boundary between two thread blocks?
- Explain the dependency using the concept of "read-after-write".
- Why is this a race condition?

Level 5: Synchronization

Objective: Coordinate threads so they do not move to the next phase early.

Task A: Add phase prints

Copy your Level 2 manual block code and

1. Print: "Thread X finished calculation" immediately after your addition loop.
2. Print: "Thread X starting verification" right after that.

Observe:

- Do some threads print "starting verification" before others finish?

Task B: Add a barrier

Insert a barrier between the two prints.

Observe:

- Do all threads now wait before verification begins?

What to write in results.txt:

- What you observed before the barrier
- What changed after the barrier